

2. Configuration minimaliste pour Apache

Lors du TP précédent, vous avez déjà pu vous rendre compte qu'Apache propose une organisation structurée et hiérarchique de sa configuration. Bien que ce soit une manière très propre de travailler, la grande quantité d'informations présente, répartie dans plusieurs fichiers, peut rendre la découverte du logiciel un peu complexe

Nous vous proposons donc dans un premier temps de travailler sur base d'une configuration minimaliste pour comprendre les directives principales.

2.1. Vérification de l'intégrité de votre labo

Nous avons constaté que, dans certaines situations, le laboratoire GNS3 supprimait de manière impromptue certains répertoires, ce qui empêche Apache de fonctionner correctement. Pour éviter ce cas de figure, voici la procédure qu'il vous faudra appliquer. Elle est également disponible sur Moodle.

Avant de commencer un exercice, vérifiez sur la machine web si les répertoires ci-dessous existent :

/etc/apache2/mods-enabled

/etc/apache2/sites-enabled

Si ce n'est pas le cas, créez-le et activez les modules qui auraient du l'être au démarrage, en effectuant les commandes suivantes :

mkdir /etc/apache2/mods-enabled

mkdir /etc/apache2/sites-enabled

a2enmod mpm_prefork

a2enmod authz_core

a2enmod dir

a2enmod mime

2.2. Mise en place de la config Apache minimaliste

La configuration par défaut d'Apache est un peu verbeuse. Nous allons donc commencer par travailler sur une configuration simplifiée afin de vous permettre de prendre vos marques.

Pour cela, effectuez une copie du fichier de configuration principal **apache2.conf** dans un fichier de backup appelé **apache2.conf_bck**.

Effacez le contenu du apache2.conf original, et insérez à la place les lignes suivantes :

```
Listen 80

ServerName www.woodytoys.lab


IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf


# These need to be set in /etc/apache2/envvars

User ${APACHE_RUN_USER}

Group ${APACHE_RUN_GROUP}


ErrorLog /var/log/apache2/error.log

LogFormat "%h %l %u %t \"%r\" %>s %O \"%{Referer}i\" \"%{User-Agent}i\"" combined
CustomLog /var/log/apache2/access.log combined

LogLevel debug
```

2.3. Répertoires Apache

Plusieurs répertoires sont impliqués dans le fonctionnement d'Apache. Examinez-les soigneusement. Il s'agit :

du répertoire **/etc/apache2/**, qui contient les fichiers de configuration. Nous commencerons ici par nous concentrer sur le fichier **apache2.conf**

du répertoire contenant par défaut les fichiers que le serveur web met à disposition des clients (pages statiques html et scripts cgi) : **/var/www**

/var/www/html/ : contient les pages du site par défaut

/var/www/cgi-bin/ : contient les scripts cgi

/var/www/error/ : contient les pages affichées en cas d'erreur — et d'autres répertoires à découvrir par vous-même.

2.4. Syntaxe des fichiers de configuration

Les fichiers de configuration d'Apache contiennent des directives, à raison de une par ligne. Il est possible de limiter la portée de ces directives en les limitant à des sections, qui sont délimitées par des tags ouvrant et fermant (**ex : <Directory>...</Directory>, <VirtualHost>...</VirtualHost>, ...**).

Les commentaires commencent par un dièse #.

Vous pouvez utiliser la commande **apache2ctl** pour vérifier la syntaxe de votre fichier de configuration. Avec l'option **-S**, cette même commande vous produira un résumé de votre configuration.

La sortie que vous obtenez :

```
VirtualHost configuration:
ServerRoot: "/etc/apache2"
Main DocumentRoot: "/var/www/html"
Main ErrorLog: "/var/log/apache2/error.log"
Mutex default: dir="/var/run/apache2/" mechanism=default
Mutex watchdog-callback: using_defaults
PidFile: "/var/run/apache2.pid"
Define: DUMP_VHOSTS
Define: DUMP_RUN_CFG
User: name="www-data" id=33
Group: name="www-data" id=33
```

Explication détaillée ligne par ligne

VirtualHost configuration:

Cette section devrait afficher vos VirtualHosts **si vous en avez configuré**. Ici, elle est vide, cela signifie **qu'aucun VirtualHost personnalisé n'est chargé**.

Apache utilise donc **le vhost par défaut**, situé dans :

```
/etc/apache2/sites-enabled/000-default.conf
```

ServerRoot: `"/etc/apache2"`

C'est le dossier principal contenant :

- les configurations (apache2.conf)
- les modules (mods-enabled)
- les sites (sites-enabled)
- les sites disponibles (sites-available)

Main DocumentRoot: `"/var/www/html"`

C'est **le dossier utilisé pour servir le site par défaut** d'Apache. Si vous ne configurez aucun VirtualHost, c'est ce site qui sera affiché.

Main ErrorLog: `"/var/log/apache2/error.log"`

Fichier où Apache écrit toutes ses erreurs.

Utile si vous déboguez :

```
tail -f /var/log/apache2/error.log
```

Mutex (verrous internes)

Lignes internes utilisées par Apache pour son bon fonctionnement. Normalement, rien à modifier ici.

PidFile Fichier contenant le PID (identifiant) du processus Apache.

Define: `DUMP_VHOSTS / DUMP_RUN_CFG`

Variables internes utilisées par Apache lorsque vous utilisez l'option **-S**.

User / Group : `www-data`

Apache tourne avec l'utilisateur et groupe :

www-data : 33

2.5. Découverte des directives principales et test de la configuration

Parcourez le fichier **/etc/apache2/apache2.conf**, et consultez la documentation officielle d'Apache pour expliquer ce que fait chaque directive. Vous pouvez soit lire la [documentation générale](#), ou bien aller directement à la [documentation des directives](#).

Résumé simple

Directive	Rôle
Listen 80	Apache écoute sur le port HTTP
ServerName	Nom d'hôte du serveur principal
IncludeOptional	Charge modules & configs si présents
User / Group	Détermine l'utilisateur système d'Apache
ErrorLog	Chemin du fichier d'erreurs
LogFormat	Définit comment Apache formate les logs
CustomLog	Fichier des requêtes entrantes
LogLevel debug	Niveau de détail maximal des logs

Consultez également le fichier **/etc/apache2/envvars** :

que contient-il ?

En quoi est-il important pour le fichier **/etc/apache2/apache2.conf** ?

Le fichier **envvars** est utilisé par

- la commande **apache2ctl**,
- les scripts de démarrage **/etc/init.d/apache2**,
- certains scripts de logrotate,

pour définir **les variables d'environnement nécessaires à Apache** sur les distributions Debian/Ubuntu.

Ces variables sont ensuite utilisées **dans la configuration Apache**, notamment dans **apache2.conf**.

Analyse du contenu du fichier /etc/apache2/envvars

unset HOME

Apache supprime la variable HOME pour éviter :

- des comportements imprévisibles
- que les scripts CGI utilisent le mauvais répertoire home

Gestion de plusieurs instances Apache

```
if [ "${APACHE_CONFDIR##/etc/apache2-}" != "${APACHE_CONFDIR}" ] ; then
    SUFFIX="-${APACHE_CONFDIR##/etc/apache2-}"
else
    SUFFIX=
fi
```

Permet d'exécuter plusieurs instances d'Apache (ex: apache2-instance1, apache2-instance2).
Le suffixe s'ajoute aux chemins (ex: /var/run/apache2-instance1/).

Définition des variables APACHE_RUN_USER et APACHE_RUN_GROUP

```
export APACHE_RUN_USER=www-data
```

```
export APACHE_RUN_GROUP=www-data
```

Ces variables sont critiques. Elles définissent l'utilisateur système sous lequel Apache tourne.

Ce sont EXACTEMENT celles utilisées dans votre fichier **apache2.conf** :

```
User ${APACHE_RUN_USER}
```

```
Group ${APACHE_RUN_GROUP}
```

Sans **envvars**, Apache ne saurait pas quels **user/group** utiliser.

Le serveur tournerait en **root** → **extrêmement dangereux**.

Fichiers PID, Lock, Run directories

```
export APACHE_PID_FILE=/var/run/apache2$SUFFIX/apache2.pid
```

```
export APACHE_RUN_DIR=/var/run/apache2$SUFFIX
```

```
export APACHE_LOCK_DIR=/var/lock/apache2$SUFFIX
```

Rôle :

- **PID_FILE** : contient l'ID du processus Apache principal
- **RUN_DIR** : répertoire de sockets UNIX, pipes, runtime
- **LOCK_DIR** : répertoire de verrous internes d'Apache

Utilisés par le script `/etc/init.d/apache2`, pour savoir comment **démarrer/arrêter** le service.

Logs

```
export APACHE_LOG_DIR=/var/log/apache2$SUFFIX
```

Définit le répertoire log par défaut pour Apache.

C'est ce répertoire utilisé dans `apache2.conf` :

```
ErrorLog /var/log/apache2/error.log
```

```
CustomLog /var/log/apache2/access.log combined
```

Grâce à envvars, **les chemins de log sont homogènes** dans toute la configuration et les scripts systèmes.

Locale

```
export LANG=C
```

Définit la locale pour les modules Apache (comme mod_dav).

Options avancées (commentées)

Exemples :

```
#APACHE_ULIMIT_MAX_FILES='ulimit -n 65536'
```

```
#export APACHE_ARGUMENTS=""
```

- **ulimit** : augmenter le nombre maximum de fichiers ouverts par Apache
- **APACHE_ARGUMENTS** : ajouter des arguments à l'exécutable Apache
- mode debug pour les mainteneurs : `APACHE2_MAINTSCRIPT_DEBUG=1`

Pourquoi ce fichier est-il important pour `apache2.conf` ?

Parce que **apache2.conf** utilise **des variables définies dans envvars**.

Sans envvars :

- Apache ne saurait pas quel utilisateur utiliser
 - Le serveur pourrait refuser de démarrer
 - Les permissions des fichiers de log pourraient devenir incorrectes
-

Ensuite, démarrez le serveur en utilisant la commande **apache2ctl start**. Lancez une capture Wireshark sur le lien menant au serveur.

Vérifiez que le processus s'est lancé correctement et est à l'écoute sur les bons ports avec les outils habituels : **ps -A** et **netstat -nltpu**.

Depuis le client, vérifiez que vous obtenez bien une page HTML en utilisant le navigateur links :

links **http://www.woodytoys.lab/index.html**

Observez le trafic HTTP échangé. Regardez également dans les logs (**/var/log/apache2**) pour voir si les requêtes effectuées ont bien été enregistrées

3. Exercice de configuration

Modifiez le fichier **apache2.conf** pour introduire les modifications ci-dessous à la configuration de votre serveur. Vous trouverez les directives à utiliser dans la documentation Apache.

Faites en sorte que votre fichier de configuration soit clair, ne contienne que les lignes nécessaires ainsi que les commentaires adéquats. Testez et validez chaque fonctionnalité après l'avoir implémentée, en réfléchissant à une procédure de validation adéquate.

Faire en sorte que si aucun fichier n'est spécifié dans l'URL, le serveur renvoie le fichier **index.html** par défaut. Cela permettra au client d'obtenir une page web lorsqu'il navigue directement sur **http://www.woodytoys.lab**. (Astuce : Si la directive est refusée, vous devrez probablement charger le **module dir** : `a2enmod dir`).

sudo a2enmod dir

Modifier le répertoire de stockage des documents web (DocumentRoot) pour que le nouveau répertoire soit **/home/user/html**. Pour tester : N'oubliez pas de créer le répertoire en question et d'y ajouter un fichier html!

```
mkdir -p /home/user/html
```

```
echo "<h1>Test OK</h1>" > /home/user/html/index.html
```

```
chmod -R 755 /home/user/html
```

```
mkdir /home/user/html
```

Crée **un seul dossier**

Échoue si le dossier existe déjà

Échoue aussi si un dossier parent manque

```
mkdir -p /home/user/html
```

Fait **trois choses utiles** :

1. **Crée tous les dossiers parents** si nécessaire, (ex : /home, /home/user ... si manquants)
2. **Ne donne pas d'erreur si le dossier existe déjà**, c'est silencieux, propre, idéal dans des scripts
3. **Assure que le chemin final existe**, garanti, même si des dossiers intermédiaires manquent

Conclusion : **mkdir -p** = créer le chemin complet sans risque d'erreur.

```
chmod 755 /home/user/html
```

Change les permissions **uniquement sur le dossier lui-même**

```
chmod -R 755 /home/user/html
```

Applique les permissions :

- au dossier /home/user/html
- **à tous les sous-dossiers**
- **à tous les fichiers dedans**
- et **aux sous-sous-dossiers**, etc.

En résumé : **chmod -R** = je change les permissions sur toute l'arborescence.

Spécifiez une adresse email pour l'administrateur (par ex. la vôtre)

Indiquer que les fichiers envoyés par défaut pour chaque répertoire sont les fichiers **test.html** au lieu de **index.html**

Le serveur ne doit accepter de requêtes que de la part du directeur

Le serveur écoute sur les ports 80 et 8080

Le fichier final (apache2.conf) doit contenir tout ce qui existait + ces nouvelles directives.

Première config.(fait par moi)

Définir index.html par défaut

DirectoryIndex index.html

Nouveau répertoire racine

DocumentRoot /home/user/html

<Directory /home/user/html>

Require all granted

</Directory>

Adresse e-mail de l'administrateur du serveur

ServerAdmin example@gmail.com

Tous les sous-répertoires : test.html par défaut

<Directory /home/user/html/>

DirectoryIndex test.html

Require all granted

</Directory>

Seul le directeur peut accéder

Require ip 192.168.0.000

Ports d'écoute du serveur

Listen 80

Listen 8080

Deuxième config (fait par l'IA).

Ports sur lesquels le serveur écoute

Listen 80

Listen 8080

Adresse e-mail de l'administrateur

ServerAdmin exemple@gmail.com

Répertoire racine des documents web

DocumentRoot /home/user/html

<Directory /home/user/html>

Si aucun fichier n'est spécifié à la racine, servir index.html

DirectoryIndex index.html

Options de base pour ce répertoire

Options Indexes FollowSymLinks

AllowOverride None

Seul le directeur peut accéder

Require ip 192.168.0.000

```
</Directory>
```

```
# -----
```

```
# Tous les sous-répertoires : fichiers par défaut test.html
```

```
# -----
```

```
<Directory /home/user/html/>
```

```
# Servir test.html par défaut pour les sous-répertoires
```

```
DirectoryIndex test.html index.html
```

```
Options Indexes FollowSymLinks
```

```
AllowOverride None
```

```
# Seul le directeur peut accéder
```

```
Require ip 192.168.0.000
```

```
</Directory>
```

Le fichier final (**apache2.conf**) doit contenir tout ce qui existait + ces nouvelles directives.

4. Configuration Apache hiérarchique

A présent que vous avez pu manipuler les bases des directives Apache, nous allons revenir à la version **“hiérarchique”** des configurations.

Dans le répertoire **/etc/apache2**, rétablissez en tant que fichier de config principal le fichier **apache2.conf_bck**, qui contient la configuration habituellement fournie lors de l’installation du logiciel serveur. Pour cela, copiez-le dans le fichier **apache2.conf** :

```
cp apache2.conf_bck apache2.conf
```

Regardez le résultat de la configuration via la commande suivante :

```
apache2ctl -S
```

Enfin, prenez le temps de lire les commentaires du fichier de configuration principal : il explique comment les configurations sont structurées. Explorez également les autres fichiers disponibles, et répondez aux questions ci-dessous :

Dans quel fichier est défini le port sur lequel Apache doit écouter ?

```
# Include list of ports to listen on
```

```
Include ports.conf
```

```
/etc/apache2/ports.conf
```

Comment savoir quels modules sont activés ?

Apache inclut les modules via les dossiers `mods-enabled/*.load` et `mods-enabled/*.conf` :

```
IncludeOptional mods-enabled/*.load
```

```
IncludeOptional mods-enabled/*.conf
```

Commande pratique pour vérifier les modules activés :

```
apache2ctl -M
```

Pour activer/désactiver un module :

```
a2enmod <nom_module> # activer
```

```
a2dismod <nom_module> # désactiver
```

Qui est l'utilisateur unix utilisé par Apache, et à quel endroit est-il configuré ?

Déclaré dans `/etc/apache2/envvars` mais référencé dans `apache2.conf` :

```
User ${APACHE_RUN_USER}
```

```
Group ${APACHE_RUN_GROUP}
```

Par défaut sur Debian : **www-data** pour l'utilisateur et le groupe.

C'est cet utilisateur qui exécute le serveur Apache et accède aux fichiers.

A quel endroit spécifie-t-on le port à utiliser en cas d'activation de SSL ?

Fichier : **sites-available/default-ssl.conf**

Dans ce fichier, le VirtualHost pour SSL est généralement défini ainsi :

```
<VirtualHost _default_:443>

    SSLEngine on

    SSLCertificateFile /etc/ssl/certs/ssl-cert-snakeoil.pem

    SSLCertificateKeyFile /etc/ssl/private/ssl-cert-snakeoil.key

</VirtualHost>
```

Le port **443** peut également être défini globalement dans **ports.conf** :

```
Listen 443
```

5. Virtual Hosts

Apache permet à un serveur de gérer plusieurs sites, appelés **virtual hosts**. Pour pouvoir lier une requête au site correspondant, il peut se baser sur trois éléments :

L'hôte utilisé (nom DNS renseigné dans le champ Host du header HTTP)

Le port utilisé (ex : un site sur le port 80, un autre sur le port 81)

L'adresse IP utilisée, si la machine en possède plusieurs.

Pour pouvoir gérer cela, Apache propose de séparer la configuration de chaque site dans un fichier dédié. Vous trouverez une config de site par défaut (**000-default.conf**) dans le répertoire **sites-available**. Prenez-en connaissance.

Nous allons nous baser sur cet exemple pour configurer deux virtual hosts : Un sur **www.woodytoys.lab**, et un autre sur **blog.woodytoys.lab**.

Pour cela, effectuez les étapes suivantes :

Allez modifier le fichier de zone dans le SOA pour ajouter un **RR CNAME** permettant de faire pointer le nom **blog.woodytoys.lab** vers **www.woodytoys.lab**. Cela permettra au client de trouver l'adresse IP du site **blog.woodytoys.lab**.

Un **CNAME** (Canonical Name) est un type d'enregistrement DNS qui permet de dire :

Ce nom de domaine est un alias d'un autre nom de domaine.

Autrement dit : il ne pointe pas directement vers une adresse IP, mais vers un autre nom DNS, qui lui pointe vers l'IP.

Exemple simple

Tu ajoutes ceci dans ta zone :

blog IN CNAME www.woodytoys.lab.

Cela signifie : **blog.woodytoys.lab** = alias de www.woodytoys.lab

Donc si un client cherche l'IP de **blog.woodytoys.lab**, le DNS va automatiquement renvoyer :

- d'abord : **blog** → www.woodytoys.lab
- puis : l'IP attachée à www.woodytoys.lab

Ouvrez le fichier de zone de woodytoys.lab

Dans ma vm le fichier se trouve ici :

/etc/bind/woodytoys.lab

Sous les enregistrements A, ajoute simplement :

blog IN CNAME www.woodytoys.lab.

Avec un point final à la fin (lab.), sinon ce sera interprété comme un nom relatif.

Incrémente le Serial du SOA

Par exemple, si tu as :

```
2 ; Serial
```

Tu mets :

```
3 ; Serial
```

Obligatoire, sinon Bind ne rechargera pas les modifications !

Créez un répertoire pour chaque site (www et blog) :

Dans le répertoire des logs (/var/log/apache2/www et /var/log/apache2/blog)

```
mkdir -p /var/log/apache2/www
```

```
mkdir -p /var/log/apache2/blog
```

Dans le répertoire contenant les fichiers HTML (/var/www/html/www et /var/www/html/blog).
Créez un fichier index.html dans chacun de ces répertoires, avec un contenu différent (ex :
Bienvenue sur www et *Bienvenue sur le blog*)

```
mkdir -p /var/www/html/www
```

```
mkdir -p /var/www/html/blog
```

Page du site principal

```
nano /var/www/html/www/index.html
```

```
<!DOCTYPE html>

<html lang="fr">

<head>

  <meta charset="UTF-8">

  <title>Site WWW - WoodyToys</title>

</head>

<body>

  <h1>Bienvenue sur le site WWW de WoodyToys</h1>

  <p>Ceci est la page principale du site www.woodytoys.lab.</p>

  <p>Ce site est fièrement servi par un Apache dompté par Jeremie, maître des câbles
et conquérant des configurations</p>

</body>

</html>
```

Page du blog

```
nano /var/www/html/blog/index.html
```

```
<!DOCTYPE html>

<html lang="fr">

<head>

  <meta charset="UTF-8">

  <title>Blog - WoodyToys</title>

</head>

<body>

  <h1>Bienvenue sur le blog de WoodyToys</h1>

  <p>Ceci est la page d'accueil du site blog.woodytoys.lab.</p>

  <p>Ce blog est tenu par Jeremie : bricoleur, codeur et dompteur de Raspberry Pi</p>

</body>

</html>
```

Créez deux copies du fichier 000-default.conf, appelées www-woodytoys-lab.conf et blog-woodytoys-lab.conf. Adaptez ces deux fichiers :

```
cp /etc/apache2/sites-available/000-default.conf /etc/apache2/sites-available/www-woodytoys-lab.conf
```

```
cp /etc/apache2/sites-available/000-default.conf /etc/apache2/sites-available/blog-woodytoys-lab.conf
```

Au niveau de la directive **ServerName**, qui servira à Apache pour associer chaque requête au virtual host correspondant

Au niveau du répertoire contenant les documents HTML du site

Au niveau du répertoire de log.

Fichier www-woodytoys-lab.conf

```
nano /etc/apache2/sites-available/www-woodytoys-lab.conf
```

```
<VirtualHost *:80>

    ServerName www.woodytoys.lab

    DocumentRoot /var/www/html/www

    ErrorLog /var/log/apache2/www/error.log

    CustomLog /var/log/apache2/www/access.log combined

</VirtualHost>
```

Fichier blog-woodytoys-lab.conf

```
nano /etc/apache2/sites-available/blog-woodytoys-lab.conf
```

```
<VirtualHost *:80>

    ServerName blog.woodytoys.lab

    DocumentRoot /var/www/html/blog

    ErrorLog /var/log/apache2/blog/error.log

    CustomLog /var/log/apache2/blog/access.log combined

</VirtualHost>
```

Activez les deux virtual host avec la commande **a2ensite**.

```
a2ensite www-woodytoys-lab
```

```
a2ensite blog-woodytoys-lab
```

Vérifiez la configuration avec

```
apache2ctl -S
```

Redémarrez le serveur Apache, et testez les deux sites depuis le client.

apache2ctl start

links <http://www.woodytoys.lab>

links <http://blog.woodytoys.lab>